

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Описание предметной области	6
1.2 Требования к системе	7
1.3 Входные/выходные данные, ограничения	9
ГЛАВА 2. РАЗРАБОТКА КОНСОЛЬНОГО ПРИЛОЖЕНИЯ	11
2.1 Модель данных	11
2.2 Разработка консольного приложения	12
2.2.1 Пользовательский интерфейс (консольное меню)	13
2.2.2 Работа с CSV-файлом	14
2.2.3 Реализация функционала приложения	16
ГЛАВА 3. ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ	19
3.1 Тестовые сценарии	19
3.2 Результаты тестирования	22
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ	25
ПРИЛОЖЕНИЕ	27
ПРИЛОЖЕНИЕ А	28
ПРИЛОЖЕНИЕ Б	39
ПРИЛОЖЕНИЕ В	46

ВВЕДЕНИЕ

В современных условиях развитие информационных технологий оказывает значительное влияние на организацию учёта, хранения и обработки данных в различных предметных областях. Автоматизация учётных процессов позволяет сократить время обработки информации, уменьшить количество ошибок, связанных с ручным вводом данных, а также повысить удобство поиска, анализа и хранения сведений.

Актуальность темы курсовой работы обусловлена необходимостью разработки простых и надёжных программных средств для учёта автобусных рейсов. Пассажирский транспорт является важной частью городской и межгородской инфраструктуры, а учёт рейсов и их параметров — ключевой задачей для транспортных организаций. Некорректное ведение информации о рейсах может привести к неудобствам для пассажиров, задержкам и финансовым потерям [1]. Кроме того, автоматизация процессов хранения и обработки транспортных данных соответствует современным тенденциям цифровизации отрасли [2].

Разрабатываемое консольное приложение предназначено для учёта автобусных рейсов. Оно позволяет добавлять новые рейсы, просматривать список рейсов, выполнять поиск, редактирование и удаление данных. Приложение поддерживает сортировку, расчёт агрегированных показателей и сохранение информации в CSV-файл. Язык Kotlin поддерживает объектно-ориентированный подход, работу с классами, коллекциями и средствами обработки данных, что делает его подходящим для разработки учебного консольного приложения [3].

Целью курсовой работы является разработка консольного приложения на языке Kotlin для учёта, хранения и обработки данных об автобусных рейсах с использованием принципов объектно-ориентированного программирования.

Для достижения поставленной цели необходимо решить следующие задачи:

- выполнить анализ предметной области учёта автобусных рейсов;
- определить функциональные и нефункциональные требования к разрабатываемой системе;
- определить входные и выходные данные приложения, а также основные ограничения;
- разработать модель данных для хранения информации об автобусном рейсе;
- реализовать консольное меню приложения;
- реализовать функции добавления, просмотра, поиска, редактирования и удаления рейсов;
- реализовать сортировку и расчёт агрегированных показателей;
- реализовать сохранение и загрузку данных с использованием CSV-файла;
- выполнить тестирование разработанного консольного приложения.

Объектом исследования является процесс учёта автобусных рейсов. Предметом исследования является консольное приложение на языке Kotlin, предназначенное для хранения, обработки и анализа данных об автобусных рейсах.

Курсовая работа состоит из введения, трёх глав, заключения, списка использованных источников и приложений. В первой главе рассматривается предметная область учёта автобусных рейсов, определяются требования к системе, входные и выходные данные. Во второй главе описывается разработка консольного приложения, модель данных, пользовательское меню, работа с CSV-файлом и реализация основного функционала. В третьей главе приводятся тестовые сценарии и результаты тестирования разработанного приложения.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Описание предметной области

Автобусный транспорт является одним из ключевых видов общественного транспорта, обеспечивающим перемещение пассажиров как внутри городов, так и между населёнными пунктами. Организация пассажирских перевозок связана с ведением учёта рейсов — регулярных маршрутов, по которым осуществляется движение автобусов по установленному расписанию.

Учёт автобусных рейсов включает хранение сведений о номере маршрута, пунктах отправления и назначения, времени отправления, стоимости билетов, количестве мест в транспортном средстве и текущем статусе рейса. Эти данные необходимы для планирования перевозок, информирования пассажиров и анализа финансовых показателей транспортного предприятия [1].

При ручном ведении учёта рейсов могут возникать различные проблемы: потеря данных, дублирование записей, ошибки при вводе числовых параметров, сложность поиска нужного рейса, а также отсутствие быстрой сортировки и фильтрации информации. Использование программного средства позволяет снизить вероятность таких ошибок и повысить удобство работы с данными.

Основной сущностью предметной области является автобусный рейс. В рамках разрабатываемой системы рейс включает следующие данные:

- идентификатор рейса — уникальный числовой код для однозначного определения рейса в системе;
- номер маршрута — обозначение маршрута (например, 101, А52);
- пункт отправления — наименование населённого пункта или остановки, откуда начинается маршрут;
- пункт назначения — наименование конечной остановки маршрута;
- время отправления — плановое время выезда в формате чч:мм;
- цена билета — стоимость одного билета в рублях;
- общее количество мест — вместимость автобуса;
- количество свободных мест — число незанятых мест на текущий момент;

– статус рейса — текущее состояние рейса.

В приложении предусмотрены следующие статусы рейсов: «Активен», «Отменён», «Задержан», «Завершён». Использование статусов позволяет разделять рейсы по текущему состоянию и применять их при поиске и фильтрации данных.

Таким образом, предметная область учёта автобусных рейсов представляет собой совокупность процессов по хранению, обработке и анализу сведений о пассажирских маршрутах. Автоматизация данной предметной области позволяет повысить надёжность хранения данных, ускорить выполнение операций и обеспечить удобный доступ к информации.

1.2 Требования к системе

Разрабатываемая система должна представлять собой консольное приложение на языке Kotlin. Kotlin является современным языком программирования, который поддерживает объектно-ориентированный подход, работу с классами, коллекциями, функциями и механизмами обработки данных [3]. Данные возможности позволяют использовать его для разработки учебного консольного приложения, предназначенного для обработки записей выбранной предметной области.

Взаимодействие пользователя с программой должно осуществляться через циклическое текстовое меню. Пользователь выбирает пункт меню, после чего программа выполняет соответствующую операцию и возвращается к главному меню.

К функциональным требованиям системы относятся:

- добавление нового автобусного рейса;
- отображение всех автобусных рейсов;
- поиск рейсов по пункту отправления, назначения, номеру маршрута или статусу;
- редактирование существующего рейса по идентификатору;
- удаление рейса по идентификатору;

- сортировка рейсов по различным полям;
- расчёт агрегированных показателей по ценам и количеству мест;
- сохранение данных в CSV-файл;
- загрузка данных из CSV-файла;
- обработка некорректного ввода пользователя.

Добавление рейса должно предусматривать ввод номера маршрута, пунктов отправления и назначения, времени отправления, цены, количества мест и статуса. Каждому рейсу должен присваиваться уникальный идентификатор автоматически.

Отображение рейсов должно выполняться в табличном виде. Пользователь должен видеть идентификатор, номер маршрута, пункты отправления и назначения, время, цену, количество мест и статус каждого рейса.

Поиск должен позволять находить записи по текстовым полям и статусу. Если подходящие записи отсутствуют, программа должна выводить соответствующее сообщение. Редактирование и удаление должны выполняться по идентификатору рейса. Если запись с указанным идентификатором отсутствует, система должна сообщить пользователю об ошибке.

К нефункциональным требованиям системы относятся:

- приложение должно работать в консольном режиме;
- интерфейс должен быть понятным для пользователя;
- программа должна использовать циклическое меню;
- данные во время работы программы должны храниться в коллекции;
- для описания рейса должен использоваться data class;
- сохранение данных должно выполняться в файл формата CSV;
- программа должна корректно реагировать на неверный ввод пользователя;
- приложение не должно завершаться аварийно при ошибках ввода.

Для хранения набора рейсов во время выполнения программы используется изменяемая коллекция `MutableList`, так как коллекции Kotlin предназначены для хранения и обработки групп объектов [6]. Для описания рейса

применяется `data class`, поскольку данный тип класса удобен для представления объектов, основная задача которых заключается в хранении данных [5].

1.3 Входные/выходные данные, ограничения

Входными данными приложения являются сведения, которые пользователь вводит через консоль. К таким данным относятся:

- выбранный пункт главного меню;
- номер маршрута;
- пункт отправления;
- пункт назначения;
- время отправления;
- цена билета;
- общее количество мест;
- количество свободных мест;
- статус рейса;
- идентификатор рейса для редактирования или удаления;
- параметры поиска и сортировки.

Также входными данными может являться CSV-файл, из которого приложение загружает ранее сохранённые рейсы. CSV-файл содержит данные об идентификаторе, номере маршрута, пунктах, времени, цене, количестве мест и статусе рейса.

Выходными данными приложения являются:

- сообщения о результате выполнения операций;
- список автобусных рейсов в табличном виде;
- найденные и отсортированные записи;
- агрегированные показатели по ценам и местам;
- сообщения об ошибках;
- CSV-файл с сохранёнными данными.

В программе предусмотрены ограничения, направленные на повышение корректности данных. Идентификатор рейса должен быть положительным

целым числом и не должен повторяться. Номер маршрута, пункты отправления и назначения не должны быть пустыми строками. Цена должна быть неотрицательным числом. Количество свободных мест не должно превышать общее количество мест. Статус рейса должен выбираться из заранее заданного списка допустимых значений: «Активен», «Отменён», «Задержан», «Завершён».

При некорректном вводе пользователя программа должна сообщать о причине ошибки и возвращаться к дальнейшей работе с меню. Для этого в приложении используются проверки вводимых значений и обработка исключительных ситуаций [7].

В качестве основного способа хранения данных во время работы приложения используется коллекция `MutableList`. Для долговременного хранения используется CSV-файл. Такой формат выбран из-за простоты структуры и удобства последующей обработки данных.

Таким образом, в ходе анализа предметной области были определены основные характеристики разрабатываемой системы: перечень функциональных и нефункциональных требований, состав входных и выходных данных, а также ограничения на вводимые значения. Полученные результаты служат основой для проектирования и реализации консольного приложения.

ГЛАВА 2. РАЗРАБОТКА КОНСОЛЬНОГО ПРИЛОЖЕНИЯ

2.1 Модель данных

Для разработки консольного приложения была определена основная сущность предметной области — автобусный рейс. Рейс содержит сведения, необходимые для учёта и обработки информации: идентификатор, номер маршрута, пункты отправления и назначения, время отправления, цену, количество мест и статус.

В языке Kotlin для хранения набора связанных данных удобно использовать классы. В разработанном приложении модель автобусного рейса реализована с помощью data class, так как такой тип класса предназначен для описания объектов, основная задача которых заключается в хранении данных [5]. Модель данных представлена в листинге 2.1.

Листинг 2.1 — Модель данных автобусного рейса

```
import java.io.File

data class BusRoute(
    val id: Int,
    var routeNumber: String,
    var departure: String,
    var destination: String,
    var departureTime: String,
    var price: Double,
    var seatsTotal: Int,
    var seatsAvailable: Int,
    var status: String
)

val routes = mutableListOf<BusRoute>()
val allowedStatuses = listOf("Активен", "Отменён", "Задержан", "Завершён")
var nextId = 1
val csvFile = "routes.csv"
```

Поле `id` используется как уникальный идентификатор рейса. Поля `routeNumber`, `departure`, `destination`, `departureTime` и `status` являются строковыми. Поле `price` имеет тип `Double`, поскольку цена может быть представлена числом с дробной частью. Поля `seatsTotal` и `seatsAvailable` имеют тип `Int`.

Для хранения рейсов во время работы программы используется изменяемая коллекция `MutableList`. Коллекции в Kotlin предназначены для хранения групп объектов и выполнения операций над ними, таких как поиск, фильтрация, сортировка и подсчёт элементов [6].

Для формирования уникальных идентификаторов применяется переменная `nextId`. При добавлении нового рейса значение `nextId` присваивается полю `id`, после чего увеличивается на единицу. После загрузки данных из CSV-файла значение `nextId` пересчитывается на основе максимального идентификатора.

Также в программе определён список допустимых статусов: `allowedStatuses`. Использование заранее заданного списка позволяет ограничить ввод пользователя и избежать появления некорректных значений.

Для подсчёта рейсов по статусам используется структура `Map`. В языке Kotlin `Map` применяется для хранения пар «ключ — значение», что удобно для представления количества рейсов по каждому статусу [6].

Таким образом, разработанная модель данных на основе `data class BusRoute` охватывает все необходимые атрибуты автобусного рейса и обеспечивает основу для реализации операций добавления, редактирования, поиска и отображения данных.

2.2 Разработка консольного приложения

Программа представляет собой консольное приложение с циклическим меню. Пользователь взаимодействует с системой путём ввода числовых команд в консоль. После выполнения выбранного действия приложение снова выводит главное меню, пока пользователь не выберет пункт выхода. Основная функция программы представлена в листинге 2.2.

Листинг 2.2 — Функция `main` (основная функция программы)

```
fun main() {
    loadFromCsv(showSuccessMessage = false, missingFileIsError = false)
    var choice: Int?
    do {
        println()
        println("=== СИСТЕМА УЧЁТА АВТОБУСНЫХ РЕЙСОВ ===")
        println("Главное меню:")
    }
```

```

println("1. Ввод/вывод данных")
println("2. Отобразить все рейсы")
println("3. Найти рейс")
println("4. Редактировать рейс")
println("5. Удалить рейс")
println("6. Агрегированные показатели")
println("7. Сортировка рейсов")
println("8. Выход")
choice = readInt("Введите выбор: ")
when (choice) {
    1 -> inputOutputMenu()
    2 -> showRoutes()
    3 -> findRoutes()
    4 -> editRoute()
    5 -> deleteRoute()
    6 -> showStats()
    7 -> sortRoutes()
    8 -> println("Выход из программы...")
    else -> println("Ошибка: введите число от 1 до 8.")
}
} while (choice != 8)
}

```

При запуске программы выполняется автоматическая загрузка данных из CSV-файла. После этого запускается цикл `do while`, обеспечивающий повторное отображение меню. Выбор действия реализован с помощью конструкции `when`, которая позволяет выполнять разные блоки кода в зависимости от значения переменной [11].

2.2.1 Пользовательский интерфейс (консольное меню)

Пользовательский интерфейс приложения реализован в виде текстового консольного меню. Главное меню содержит восемь пунктов:

1. ввод/вывод данных — подменю для загрузки из файла, сохранения в файл и добавления рейса;
2. отображение всех рейсов — вывод всех записей в табличной форме;
3. найти рейс — поиск по различным критериям;
4. редактировать рейс — изменение данных рейса по ID;
5. удалить рейс — удаление записи по ID;
6. агрегированные показатели — статистика по ценам и местам;
7. сортировка рейсов — упорядочивание по выбранному полю;
8. выход — завершение работы программы.

Для считывания числовых данных от пользователя используется вспомогательная функция `readInt`, которая возвращает `null` при некорректном вводе, что позволяет реализовать проверку без выброса исключений. Аналогичная функция `readDouble` используется для считывания вещественных чисел. Функция `readString` возвращает введённую строку.

При выборе пункта «Ввод/вывод данных» отображается вложенное подменю, позволяющее загрузить данные из файла, сохранить их или добавить новый рейс. Такая структура позволяет логически сгруппировать операции ввода-вывода.

Вывод списка рейсов осуществляется в виде форматированной таблицы с использованием функции `String.format`. Каждая строка содержит данные одного рейса, выровненные по столбцам. Если список рейсов пуст, выводится соответствующее сообщение.

2.2.2 Работа с CSV-файлом

Для хранения и обмена данными о рейсах в приложении используется формат CSV (Comma-Separated Values). В качестве разделителя полей применяется символ «точка с запятой» (;), поскольку он реже встречается в текстовых значениях, чем запятая, что снижает вероятность ошибок при обработке данных.

CSV-файл содержит заголовочную строку с названиями полей, после которой располагаются записи о рейсах. Каждая строка файла соответствует одному автобусному рейсу и включает следующие данные: идентификатор, номер маршрута, пункт отправления, пункт назначения, время отправления, стоимость билета, общее количество мест, количество свободных мест и статус рейса.

Пример структуры файла:

```
id;routeNumber;departure;destination;departureTime;price;seatsTotal;seatsAvailable;status
1;101;Москва;Владимир;08:30;350.0;50;30;Активен
2;202;Санкт-Петербург;Новгород;10:00;450.0;44;20;Активен
3;305;Казань;Нижний Новгород;14:15;280.0;55;55;Завершён
```

Функция загрузки данных из CSV-файла представлена в листинге 2.3.

Листинг 2.3 — Функция loadFromCsv (загрузка из CSV-файла)

```
fun loadFromCsv(showSuccessMessage: Boolean = true,
               missingFileIsError: Boolean = true) {
    val file = File(csvFile)
    if (!file.exists()) {
        if (missingFileIsError) println("Файл '$csvFile' не найден.")
        return
    }
    routes.clear()
    var errors = 0
    val usedIds = mutableSetOf<Int>()
    file.readLines().drop(1).forEachIndexed { idx, line ->
        val parts = line.split(";")
        if (parts.size < 9) { errors++; return@forEachIndexed }
        try {
            val id = parts[0].trim().toInt()
            if (id in usedIds) { errors++; return@forEachIndexed }
            val price = parts[5].trim().toDouble()
            val status = parts[8].trim()
            if (status !in allowedStatuses) { errors++;
return@forEachIndexed }
            usedIds.add(id)
            routes.add(BusRoute(id, parts[1].trim(), parts[2].trim(),
                parts[3].trim(), parts[4].trim(), price,
                parts[6].trim().toInt(), parts[7].trim().toInt(), status))
        } catch (e: NumberFormatException) { errors++ }
    }
    nextId = (routes.maxOfOrNull { it.id } ?: 0) + 1
    if (showSuccessMessage)
        println("Загружено ${routes.size} рейс(ов). Ошибок: $errors.")
}
```

При загрузке данных выполняются следующие проверки: наличие файла, корректное количество полей в каждой строке, правильный формат числовых значений, допустимость статуса и уникальность идентификатора. Строки с ошибками пропускаются, а пользователю выводится предупреждение. После загрузки значение `nextId` пересчитывается как максимальный `id` плюс единица.

Функция сохранения данных в CSV-файл представлена в листинге 2.4.

Листинг 2.4 — Функция saveToCsv (сохранение в CSV-файл)

```
fun saveToCsv() {
    val file = File(csvFile)
    val header = "id;routeNumber;departure;destination;" +
        "departureTime;price;seatsTotal;seatsAvailable;status"
    val lines = mutableListOf(header)
    for (r in routes) {

lines.add("${r.id};${r.routeNumber};${r.departure};${r.destination};" +
        "${r.departureTime};${r.price};${r.seatsTotal};" +
```

```

        "${r.seatsAvailable};${r.status}")
    }
    file.writeText(lines.joinToString("\n"))
    println("Данные сохранены. Записей: ${routes.size}.")
}

```

Функция формирует заголовочную строку, затем создаёт строку для каждого рейса путём конкатенации полей через разделитель. Все строки объединяются в один текст и записываются в файл с помощью метода `writeText`. Работа с файлами является стандартной возможностью языка Kotlin и стандартной библиотеки [9].

2.2.3 Реализация функционала приложения

Добавление нового рейса реализовано в функции `addRoute`, представленной в листинге 2.5. Функция последовательно запрашивает у пользователя все необходимые поля, выполняет проверку каждого введённого значения и, в случае успешной валидации, добавляет новый объект `BusRoute` в коллекцию. При обнаружении некорректного значения выводится сообщение об ошибке, и функция завершает работу без добавления записи.

Листинг 2.5 — Функция `addRoute` (добавление рейса)

```

fun addRoute() {
    println("\n=== Добавление нового рейса ===")
    val routeNumber = readString("Номер маршрута: ")
    if (routeNumber.isBlank()) { println("Ошибка: номер маршрута пуст.");
return }
    val departure = readString("Пункт отправления: ")
    if (departure.isBlank()) { println("Ошибка: поле пустое."); return }
    val destination = readString("Пункт назначения: ")
    if (destination.isBlank()) { println("Ошибка: поле пустое."); return }
    val departureTime = readString("Время отправления (чч:мм): ")
    val price = readDouble("Цена билета (руб.): ")
    if (price == null || price < 0) { println("Ошибка: некорректная
цена."); return }
    val seatsTotal = readInt("Общее количество мест: ")
    if (seatsTotal == null || seatsTotal <= 0) { println("Ошибка.");
return }
    val seatsAvailable = readInt("Свободных мест: ")
    if (seatsAvailable == null || seatsAvailable < 0 || seatsAvailable >
seatsTotal)
        { println("Ошибка."); return }
    println("Статусы: ${allowedStatuses.joinToString(", ")}")
    val status = readString("Статус: ")
    if (status !in allowedStatuses) { println("Ошибка: недопустимый
статус."); return }
    routes.add(BusRoute(nextId++, routeNumber, departure, destination,
departureTime, price, seatsTotal, seatsAvailable, status))
}

```

```
        println("Рейс добавлен с ID = ${nextId - 1}.")
    }
}
```

Редактирование рейса реализовано в функции `editRoute`. Сначала пользователь вводит идентификатор рейса для редактирования. Если рейс с таким ID не найден, выводится сообщение об ошибке. В противном случае отображаются текущие данные рейса, и пользователю предлагается ввести новые значения. Если поле оставлено пустым, текущее значение сохраняется без изменений [8].

Удаление рейса выполняется функцией `deleteRoute`. Пользователь вводит идентификатор рейса. Программа ищет запись в коллекции и, если она найдена, удаляет её. Если рейс с указанным ID не существует, выводится сообщение об ошибке.

Поиск рейсов реализован в функции `findRoutes`. Пользователю предлагается выбрать критерий поиска из четырёх вариантов: по пункту отправления, по пункту назначения, по номеру маршрута и по статусу. Для текстовых полей используется поиск с игнорированием регистра (метод `contains` с параметром `ignoreCase = true`). Результаты поиска отображаются в табличном виде. Если совпадений не найдено, выводится соответствующее сообщение.

Сортировка рейсов реализована в функции `sortRoutes`. Пользователь выбирает поле для сортировки из шести вариантов: по ID, по номеру маршрута, по цене по возрастанию, по цене по убыванию, по количеству свободных мест и по статусу. Для сортировки используются функции `sortedBy` и `sortedByDescending` из стандартной библиотеки Kotlin [10].

Расчёт агрегированных показателей реализован в функции `showStats`, представленной в листинге 2.6.

Листинг 2.6 — Функция `showStats` (агрегированные показатели)

```
fun showStats() {
    println("\n=== Агрегированные показатели ===")
    if (routes.isEmpty()) { println("Нет данных."); return }
    println("Всего рейсов: ${routes.size}")
    println("Средняя цена: %.2f руб.".format(routes.map { it.price
    }.average()))
    println("Минимальная цена: %.2f руб.".format(routes.minOf { it.price
    })))
}
```

```

        println("Максимальная цена: %.2f руб.".format(routes.maxOf { it.price
    }))
    println("Сумма цен: %.2f руб.".format(routes.sumOf { it.price }))
    println("Всего мест: ${routes.sumOf { it.seatsTotal }}")
    println("Свободных мест: ${routes.sumOf { it.seatsAvailable }}")
    val byStatus: Map<String, Int> = routes.groupingBy { it.status
}.eachCount()
    println("\nПо статусам:")
    for ((s, c) in byStatus) println("  $s: $c")
}

```

Функция вычисляет общее количество рейсов, среднюю, минимальную и максимальную цену билета, сумму всех цен, общее и свободное количество мест. Также подсчитывается количество рейсов по каждому статусу с использованием метода `groupingBy` и `eachCount`, результат которых возвращается в виде объекта `Map<String, Int>`.

Для корректной обработки ошибок ввода в приложении используются вспомогательные функции `readInt`, `readDouble` и `readString`, которые возвращают `null` при невозможности преобразования введённой строки к требуемому типу. Это позволяет обрабатывать ошибочный ввод без выброса исключений и выводить пользователю понятное сообщение об ошибке [7].

Таким образом, в ходе второй главы было разработано консольное приложение на языке Kotlin, реализующее полный набор функций для учёта автобусных рейсов: добавление, просмотр, поиск, редактирование, удаление, сортировку, расчёт агрегированных показателей, а также загрузку и сохранение данных в CSV-файл. Все операции снабжены проверками корректности вводимых данных.

ГЛАВА 3. ТЕСТИРОВАНИЕ ПРИЛОЖЕНИЯ

3.1 Тестовые сценарии

Для проверки корректности работы разработанного консольного приложения были составлены тестовые сценарии, охватывающие основные функции системы учёта автобусных рейсов. В таблице 3.1 представлена сводная таблица тестовых сценариев.

Таблица 3.1 — Сводная таблица тестовых сценариев

№	Функция	Действие	Ожидаемый результат
1	Загрузка из файла	Запустить приложение с корректным CSV-файлом	Данные загружены, меню отображается
2	Загрузка из файла	Запустить приложение без CSV-файла	Меню отображается, список пуст, сообщение об отсутствии файла
3	Отображение рейсов	Выбрать пункт «Отобразить все рейсы»	Рейсы выведены в табличном виде
4	Добавление рейса	Ввести корректные данные нового рейса	Рейс добавлен с новым автоматическим ID
5	Добавление рейса	Ввести отрицательную цену	Сообщение об ошибке, рейс не добавлен
6	Добавление рейса	Оставить пустым поле пункта отправления	Сообщение об ошибке, ввод остановлен
7	Добавление рейса	Ввести недопустимый статус	Сообщение об ошибке, рейс не добавлен
8	Редактирование	Ввести существующий ID рейса	Текущие данные выведены, поля доступны для изменения
9	Редактирование	Ввести несуществующий ID рейса	Сообщение «Рейс с ID=... не найден»
10	Удаление	Ввести существующий ID рейса	Рейс удалён из коллекции
11	Удаление	Ввести несуществующий ID рейса	Сообщение «Рейс с ID=... не найден»
12	Поиск	Поиск по существующему пункту отправления	Найденные рейсы отображены в таблице

Продолжение таблицы 3.1

13	Поиск	Поиск по несуществующему значению	Сообщение об отсутствии результатов
14	Сортировка	Выбрать сортировку по цене (возрастание)	Рейсы отображены в порядке возрастания цены
15	Агрегация	Выбрать пункт «Агрегированные показатели»	Выведена статистика по ценам и местам
16	Сохранение	Выбрать «Сохранить данные в CSV»	Файл routes.csv создан/перезаписан с актуальными данными
17	Некорректный ввод в меню	Ввести букву или символ вместо числа пункта меню	Сообщение об ошибке, меню выводится повторно

Ниже приведены подробные описания наиболее важных тестовых сценариев.

Сценарий 1. Загрузка данных из корректного файла при запуске приложения

Цель: проверить корректность загрузки данных из CSV-файла при запуске приложения.

Предусловие: файл routes.csv существует и содержит корректные сведения о рейсах.

Действия пользователя:

Запустить приложение.

Дождаться вывода меню.

Выбрать пункт «Отобразить все рейсы».

Ожидаемый результат: программа успешно загружает данные из файла и отображает список рейсов.

Сценарий 2. Загрузка данных при отсутствии файла при запуске приложения

Цель: проверить работу программы при отсутствии файла с данными.

Предусловие: файл routes.csv отсутствует в каталоге программы.

Действия пользователя:

Запустить программу.

Дождаться вывода меню.

Ожидаемый результат: программа не завершает работу аварийно, выводит меню и создаёт пустой список рейсов.

Сценарий 4. Добавление нового рейса с корректными данными

Цель: проверить возможность добавления новой записи о рейсе.

Предусловие: программа запущена, коллекция доступна для изменения.

Действия пользователя:

Выбрать пункт «Ввод/вывод данных».

Выбрать «Добавить новый рейс».

Последовательно ввести: номер маршрута — «101», пункт отправления — «Москва», пункт назначения — «Владимир», время — «08:30», цену — «350», общее количество мест — «50», свободных мест — «30», статус — «Активен».

Ожидаемый результат: новый рейс успешно добавляется в список с автоматически присвоенным идентификатором.

Сценарий 8. Редактирование данных по существующему идентификатору

Цель: проверить корректность изменения данных о рейсе.

Предусловие: в списке есть запись с указанным идентификатором.

Действия пользователя:

Выбрать пункт «Редактировать рейс».

Ввести существующий идентификатор рейса.

Изменить одно или несколько полей, например цену и статус.

Оставить остальные поля пустыми.

Выбрать пункт «Отобразить все рейсы».

Ожидаемый результат: данные выбранного рейса успешно изменяются и отображаются в обновлённом виде.

Сценарий 17. Ввод некорректного пункта меню

Цель: проверить устойчивость программы к ошибочному вводу.

Предусловие: программа находится в главном меню.

Действия пользователя:

Ввести букву, символ или номер, отсутствующий в списке пунктов меню.
Ожидаемый результат: программа выводит сообщение о некорректном выборе и повторно предлагает ввести команду.

3.2 Результаты тестирования

Тестирование приложения проводилось путём последовательного выполнения всех тестовых сценариев, перечисленных в разделе 3.1. В таблице 3.2 представлены результаты тестирования.

Таблица 3.2 — Результаты выполнения тестовых сценариев

№	Ожидаемый результат	Фактический результат	Статус
1	Данные загружены	Данные загружены корректно	Пройден
2	Список пуст, нет сбоя	Меню выведено, список пуст	Пройден
3	Таблица рейсов	Таблица рейсов отображена	Пройден
4	Рейс добавлен с новым ID	Рейс добавлен с ID=1	Пройден
5	Сообщение об ошибке	Выведено «Ошибка: некорректная цена»	Пройден
6	Сообщение об ошибке	Выведено «Ошибка: поле пустое»	Пройден
7	Сообщение об ошибке	Выведено «Ошибка: недопустимый статус»	Пройден
8	Данные рейса выведены для редактирования	Текущие данные рейса выведены	Пройден
9	Сообщение «Рейс не найден»	Выведено «Рейс с ID=99 не найден»	Пройден
10	Рейс удалён	Рейс удалён, в списке не отображается	Пройден
11	Сообщение «Рейс не найден»	Выведено соответствующее сообщение	Пройден

Продолжение таблицы 3.2

12	Найденные рейсы выведены	Рейсы из «Москва» отображены	Пройден
13	Сообщение об отсутствии результатов	Выведено «не найдено»	Пройден
14	Рейсы по возрастанию цены	Сортировка выполнена корректно	Пройден
15	Статистика выведена	Все показатели рассчитаны верно	Пройден
16	Файл routes.csv создан	Файл создан, данные соответствуют	Пройден
17	Сообщение об ошибке, меню повторно	Сообщение выведено, работа продолжена	Пройден

Все 17 тестовых сценариев были выполнены успешно. Приложение корректно обрабатывает как штатные ситуации, так и некорректные входные данные. При вводе ошибочных значений программа выводит понятные сообщения об ошибках и не завершается аварийно. Загрузка и сохранение данных в CSV-файл работают корректно, включая обработку ситуации с отсутствующим файлом.

Таким образом, по результатам тестирования разработанное приложение соответствует всем функциональным и нефункциональным требованиям, сформулированным в главе 1 курсовой работы. Все реализованные функции работают в штатном режиме, а обработка ошибочного ввода обеспечивает устойчивость приложения к нештатным ситуациям.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано консольное приложение на языке Kotlin для учёта, хранения и обработки данных об автобусных рейсах.

В первой главе был проведён анализ предметной области учёта автобусных рейсов. Были определены основные сущности и их атрибуты, сформулированы функциональные и нефункциональные требования к системе, а также описаны входные и выходные данные и ограничения приложения.

Во второй главе была разработана модель данных на основе `data class` `BusRoute`, содержащего поля для хранения всех необходимых сведений о рейсе. Для хранения набора рейсов применяется изменяемая коллекция `MutableList`. Было реализовано консольное меню с восемью пунктами, а также следующий функционал: добавление, просмотр, поиск, редактирование и удаление рейсов; сортировка по различным полям; расчёт агрегированных показателей; загрузка и сохранение данных в CSV-файл; корректная обработка ошибок ввода.

В третьей главе были составлены 17 тестовых сценариев, охватывающих все основные функции приложения. По результатам тестирования все сценарии были выполнены успешно: приложение корректно обрабатывает штатные и нештатные ситуации, не завершается аварийно при некорректном вводе.

Таким образом, цель курсовой работы достигнута, все поставленные задачи выполнены. Разработанное приложение соответствует требованиям методических указаний и демонстрирует применение принципов объектно-ориентированного программирования, работы с коллекциями и файлами на языке Kotlin.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Федеральный закон «Устав автомобильного транспорта и городского наземного электрического транспорта» от 08.11.2007 N 259-ФЗ. — URL: https://www.consultant.ru/document/cons_doc_LAW_72388 (дата обращения: 28.03.2025).
2. Министерство транспорта РФ. Транспортная стратегия Российской Федерации до 2030 года. — URL: <https://mintrans.gov.ru> (дата обращения: 28.03.2025).
3. Коузен К. Kotlin. Сборник рецептов / К. Коузен ; перевод с английского А. Н. Киселева. — Москва : ДМК Пресс, 2021. — 220 с. — ISBN 978-5-97060-883-8. — URL: <https://e.lanbook.com/book/241007> (дата обращения: 01.04.2025).
4. Петросян Л. Э. Разработка мобильных приложений на языке Kotlin : учебное пособие для вузов / Л. Э. Петросян, К. В. Гусев. — 2-е изд., стер. — СПб. : Лань, 2025. — 104 с. — ISBN 978-5-507-52328-3. — URL: <https://e.lanbook.com/book/448577> (дата обращения: 01.04.2025).
5. Kotlin Documentation. Data Classes. — URL: <https://kotlinlang.org/docs/data-classes.html> (дата обращения: 01.04.2025).
6. Kotlin Documentation. Collections Overview. — URL: <https://kotlinlang.org/docs/collections-overview.html> (дата обращения: 01.04.2025).
7. Kotlin Documentation. Exceptions. — URL: <https://kotlinlang.org/docs/exceptions.html> (дата обращения: 02.04.2025).
8. Kotlin Documentation. Basic Input and Output. — URL: <https://kotlinlang.org/docs/basic-syntax.html> (дата обращения: 02.04.2025).
9. Kotlin Documentation. Reading and Writing Files. — URL: <https://kotlinlang.org/api/latest/jvm/stdlib/kotlin.io/> (дата обращения: 03.04.2025).
10. Kotlin Documentation. Collection Operations Overview. — URL: <https://kotlinlang.org/docs/collection-operations.html> (дата обращения: 03.04.2025).

11. Kotlin Documentation. Control Flow. — URL: <https://kotlinlang.org/docs/control-flow.html> (дата обращения: 04.04.2025).
12. Степанов П. В. Разработка мобильных приложений под Android на языке Kotlin : учебное пособие / П. В. Степанов, Н. А. Приходько, А. И. Запорожских. — Москва : РТУ МИРЭА, 2025. — 889 с. — ISBN 978-5-7339-2596-7. — URL: <https://e.lanbook.com/book/504855> (дата обращения: 01.04.2025).
13. Ларман К. Применение UML 2.0 и шаблонов проектирования / К. Ларман. — 3-е изд. — Москва : Вильямс, 2022. — 736 с. — ISBN 978-5-907314-71-6.
14. RFC 4180. Common Format and MIME Type for CSV Files. — URL: <https://tools.ietf.org/html/rfc4180> (дата обращения: 05.04.2025).
15. JetBrains. Kotlin Style Guide. — URL: <https://kotlinlang.org/docs/coding-conventions.html> (дата обращения: 05.04.2025).
16. Loshin D. Business Intelligence: The Savvy Manager's Guide. — 2nd ed. — Morgan Kaufmann, 2022. — 320 p.

ПРИЛОЖЕНИЕ

Приложение А. Полный листинг консольного приложения

Приложение Б. Скриншоты результатов работы консольного приложения

Приложение В. Тестовые сценарии

ПРИЛОЖЕНИЕ А

Полный листинг консольного приложения

Листинг А.1 — Файл Main.kt (полный исходный код)

```
import java.io.File

data class BusRoute(
    val id: Int,
    var routeNumber: String,
    var departure: String,
    var destination: String,
    var departureTime: String,
    var price: Double,
    var seatsTotal: Int,
    var seatsAvailable: Int,
    var status: String
)

val routes = mutableListOf<BusRoute>()
val allowedStatuses = listOf("Активен", "Отменён",
    "Задержан", "Завершён")
var nextId = 1
val csvFile = "routes.csv"

fun readInt(prompt: String): Int? {
    print(prompt)
    return readlnOrNull()?.trim()?.toIntOrNull()
}

fun readDouble(prompt: String): Double? {
    print(prompt)
    return readlnOrNull()?.trim()?.toDoubleOrNull()
}

fun readString(prompt: String): String {
```

```

        print(prompt)
        return readlnOrNull()?.trim() ?: ""
    }

    fun printSeparator() = println("-".repeat(110))

    fun printRoutes(list: List<BusRoute>) {
        if (list.isEmpty()) { println("Список рейсов пуст.");
return }
        printSeparator()
        println("%-4s | %-8s | %-16s | %-16s | %-8s | %-8s | %-6s
| %-7s | %-10s".format(
            "ID", "Маршрут", "Откуда", "Куда", "Время", "Цена",
"Мест", "Своб.", "Статус"))
        printSeparator()
        for (r in list) {
            println("%-4d | %-8s | %-16s | %-16s | %-8s | %-8.2f
| %-6d | %-7d | %-10s".format(
                r.id, r.routeNumber, r.departure, r.destination,
                r.departureTime, r.price, r.seatsTotal,
r.seatsAvailable, r.status))
        }
        printSeparator()
    }

    fun loadFromCsv(showSuccessMessage: Boolean = true,
missingFileIsError: Boolean = true) {
        val file = File(csvFile)
        if (!file.exists()) {
            if (missingFileIsError) println("Файл '$csvFile' не
найден. Начат пустой список.")
            return
        }
        routes.clear()
        var errors = 0
        val usedIds = mutableSetOf<Int>()

```

```

        file.readlines().drop(1).forEachIndexed { lineIndex, line
->
            val parts = line.split(";")
            if (parts.size < 9) { errors++; return@forEachIndexed
            }

            try {
                val id = parts[0].trim().toInt()
                if (id in usedIds) { errors++;
return@forEachIndexed }
                val price = parts[5].trim().toDouble()
                if (price < 0) { errors++; return@forEachIndexed
            }

                val seatsTotal = parts[6].trim().toInt()
                val seatsAvailable = parts[7].trim().toInt()
                val status = parts[8].trim()
                if (status !in allowedStatuses) { errors++;
return@forEachIndexed }
                usedIds.add(id)
                routes.add(BusRoute(id, parts[1].trim(),
parts[2].trim(), parts[3].trim(),
                parts[4].trim(), price, seatsTotal,
seatsAvailable, status))
            } catch (e: NumberFormatException) { errors++ }
        }
        nextId = (routes.maxOfOrNull { it.id } ?: 0) + 1
        if (showSuccessMessage) {
            println("Загружено ${routes.size} рейс(ов) из файла
'$csvFile'.")
            if (errors > 0) println("Пропущено строк с ошибками:
$errors.")
        }
    }

    fun saveToCsv() {
        val file = File(csvFile)

```

```

        val header =
            "id;routeNumber;departure;destination;departureTime;price;seatsTotal;seatsAvailable;status"

        val lines = mutableListOf(header)
        for (r in routes) {

            lines.add("${r.id};${r.routeNumber};${r.departure};${r.destination};${r.departureTime};${r.price};${r.seatsTotal};${r.seatsAvailable};${r.status}")

        }
        file.writeText(lines.joinToString("\n"))
        println("Данные успешно сохранены в файл '$csvFile'. Записей: ${routes.size}.")
    }

    fun showRoutes() {
        println("\n=== Все рейсы ===")
        printRoutes(routes)
    }

    fun addRoute() {
        println("\n=== Добавление нового рейса ===")
        val routeNumber = readString("Номер маршрута (например, 101): ")

        if (routeNumber.isBlank()) { println("Ошибка: номер маршрута не может быть пустым."); return }

        val departure = readString("Пункт отправления: ")
        if (departure.isBlank()) { println("Ошибка: пункт отправления не может быть пустым."); return }

        val destination = readString("Пункт назначения: ")
        if (destination.isBlank()) { println("Ошибка: пункт назначения не может быть пустым."); return }

        val departureTime = readString("Время отправления (чч:мм): ")
        if (departureTime.isBlank()) { println("Ошибка: время отправления не может быть пустым."); return }
    }

```

```

        val price = readDouble("Цена билета (руб.): ")
        if (price == null || price < 0) { println("Ошибка:
введите корректную неотрицательную цену."); return }
        val seatsTotal = readInt("Общее количество мест: ")
        if (seatsTotal == null || seatsTotal <= 0) {
println("Ошибка: количество мест должно быть положительным.");
return }
        val seatsAvailable = readInt("Количество свободных мест:
")
        if (seatsAvailable == null || seatsAvailable < 0 ||
seatsAvailable > seatsTotal) {
            println("Ошибка: количество свободных мест должно
быть от 0 до $seatsTotal."); return
        }
        println("Доступные статусы:
${allowedStatuses.joinToString(", ")}")
        val status = readString("Статус рейса: ")
        if (status !in allowedStatuses) { println("Ошибка:
недопустимый статус."); return }
        routes.add(BusRoute(nextId++, routeNumber, departure,
destination,
            departureTime, price, seatsTotal, seatsAvailable,
status))
        println("Рейс успешно добавлен с ID = ${nextId - 1}.")
    }

    fun editRoute() {
        println("\n=== Редактирование рейса ===")
        val id = readInt("Введите ID рейса: ") ?: run {
println("Ошибка: введите целое число."); return }
        val route = routes.find { it.id == id } ?: run {
println("Рейс с ID=$id не найден."); return }
        println("Текущие данные:"); printRoutes(listOf(route))
        println("Оставьте поле пустым для сохранения текущего
значения.")
    }

```

```

        val routeNumber = readString("Номер маршрута
[${route.routeNumber}]: ")
        if (routeNumber.isNotBlank()) route.routeNumber =
routeNumber
        val departure = readString("Пункт отправления
[${route.departure}]: ")
        if (departure.isNotBlank()) route.departure = departure
        val destination = readString("Пункт назначения
[${route.destination}]: ")
        if (destination.isNotBlank()) route.destination =
destination
        val departureTime = readString("Время отправления
[${route.departureTime}]: ")
        if (departureTime.isNotBlank()) route.departureTime =
departureTime
        val priceStr = readString("Цена билета [${route.price}]:
")
        if (priceStr.isNotBlank()) {
            val price = priceStr.toDoubleOrNull()
            if (price == null || price < 0) println("Ошибка: цена
не изменена.")
            else route.price = price
        }
        val seatsTotalStr = readString("Общее количество мест
[${route.seatsTotal}]: ")
        if (seatsTotalStr.isNotBlank()) {
            val seats = seatsTotalStr.toIntOrNull()
            if (seats == null || seats <= 0) println("Ошибка:
поле не изменено.")
            else route.seatsTotal = seats
        }
        val seatsAvailableStr = readString("Свободных мест
[${route.seatsAvailable}]: ")
        if (seatsAvailableStr.isNotBlank()) {
            val seats = seatsAvailableStr.toIntOrNull()

```

```

        if (seats == null || seats < 0 || seats >
route.seatsTotal) println("Ошибка: поле не изменено.")
        else route.seatsAvailable = seats
    }
    println("Доступные статусы:
${allowedStatuses.joinToString(", ")}")
    val status = readString("Статус [{route.status}]: ")
    if (status.isNotBlank()) {
        if (status !in allowedStatuses) println("Ошибка:
статус не изменён.")
        else route.status = status
    }
    println("Рейс с ID=$id успешно обновлён.")
}

fun deleteRoute() {
    println("\n=== Удаление рейса ===")
    val id = readInt("Введите ID рейса: ") ?: run {
println("Ошибка: введите целое число."); return }
    val route = routes.find { it.id == id } ?: run {
println("Рейс с ID=$id не найден."); return }
    routes.remove(route)
    println("Рейс с ID=$id успешно удалён.")
}

fun findRoutes() {
    println("\n=== Поиск рейсов ===")
    println("1. По пункту отправления")
    println("2. По пункту назначения")
    println("3. По номеру маршрута")
    println("4. По статусу")
    when (readInt("Выберите критерий: ")) {
        1 -> { val q = readString("Пункт отправления: ")
                val r = routes.filter {
it.departure.contains(q, ignoreCase = true) }

```

```

        if (r.isEmpty()) println("Рейсов не найдено.")
    else printRoutes(r) }

    2 -> { val q = readString("Пункт назначения: ")
        val r = routes.filter {
it.destination.contains(q, ignoreCase = true) }
        if (r.isEmpty()) println("Рейсов не найдено.")
    else printRoutes(r) }

    3 -> { val q = readString("Номер маршрута: ")
        val r = routes.filter {
it.routeNumber.contains(q, ignoreCase = true) }
        if (r.isEmpty()) println("Маршрут не найден.")
    else printRoutes(r) }

    4 -> { println("Статусы:
${allowedStatuses.joinToString(", ")}")
        val st = readString("Статус: ")
        val r = routes.filter { it.status.equals(st,
ignoreCase = true) }
        if (r.isEmpty()) println("Рейсов не найдено.")
    else printRoutes(r) }

    else -> println("Ошибка: выберите пункт от 1 до 4.")
}

}

fun sortRoutes() {
    println("\n=== Сортировка рейсов ===")
    println("1. По ID")
    println("2. По номеру маршрута")
    println("3. По цене (возрастание)")
    println("4. По цене (убывание)")
    println("5. По количеству свободных мест")
    println("6. По статусу")
    val sorted = when (readInt("Выберите поле: ")) {
        1 -> routes.sortedBy { it.id }
        2 -> routes.sortedBy { it.routeNumber }
        3 -> routes.sortedBy { it.price }
        4 -> routes.sortedByDescending { it.price }
    }
}

```

```

        5 -> routes.sortedByDescending { it.seatsAvailable }
        6 -> routes.sortedBy { it.status }
        else -> { println("Ошибка: выберите пункт от 1 до
6."); return }
    }
    printRoutes(sorted)
}

fun showStats() {
    println("\n=== Агрегированные показатели ===")
    if (routes.isEmpty()) { println("Нет данных для
анализа."); return }
    println("Всего рейсов: ${routes.size}")
    println("Средняя цена билета: %.2f
руб.".format(routes.map { it.price }.average()))
    println("Минимальная цена: %.2f руб.".format(routes.minOf
{ it.price })))
    println("Максимальная цена: %.2f
руб.".format(routes.maxOf { it.price })))
    println("Сумма всех цен: %.2f руб.".format(routes.sumOf {
it.price })))
    println("Всего мест: ${routes.sumOf { it.seatsTotal }}")
    println("Свободных мест: ${routes.sumOf {
it.seatsAvailable }}")
    println("\nКоличество рейсов по статусам:")
    val byStatus: Map<String, Int> = routes.groupingBy {
it.status }.eachCount()
    for ((status, count) in byStatus) println("  $status:
$count")
}

fun inputOutputMenu() {
    println("\n=== Ввод / Вывод данных ===")
    println("1. Загрузить данные из CSV-файла")
    println("2. Сохранить данные в CSV-файл")
    println("3. Добавить новый рейс")
}

```

```

        when (readInt("Выберите действие: ")) {
            1 -> loadFromCsv()
            2 -> saveToCsv()
            3 -> addRoute()
            else -> println("Ошибка: выберите пункт от 1 до 3.")
        }
    }

fun main() {
    loadFromCsv(showSuccessMessage = false,
missingFileIsError = false)
    var choice: Int?
    do {
        println()
        println("=== СИСТЕМА УЧЁТА АВТОБУСНЫХ РЕЙСОВ ===")
        println("Главное меню:")
        println("1. Ввод/вывод данных")
        println("2. Отобразить все рейсы")
        println("3. Найти рейс")
        println("4. Редактировать рейс")
        println("5. Удалить рейс")
        println("6. Агрегированные показатели")
        println("7. Сортировка рейсов")
        println("8. Выход")
        choice = readInt("Введите выбор: ")
        when (choice) {
            1 -> inputOutputMenu()
            2 -> showRoutes()
            3 -> findRoutes()
            4 -> editRoute()
            5 -> deleteRoute()
            6 -> showStats()
            7 -> sortRoutes()
            8 -> println("Выход из программы...")
            else -> println("Ошибка: введите число от 1 до
8.")
        }
    }
}

```

```
        }  
    } while (choice != 8)  
}
```

ПРИЛОЖЕНИЕ Б

Скриншоты результатов работы консольного приложения

Главное меню:

```
gaukhar@MacBook-Air-Zhanet: ~/курсовая/котлин % cd "/Users/gaukhar/Downloads/курсовая" && kotlinc Mainn.kt -include-runtime -d Mainn.jar && java -jar Mainn.jar

== СИСТЕМА УЧЁТА АВТОБУСНЫХ РЕЙСОВ ==
Главное меню:
1. Ввод/вывод данных
2. Отобразить все рейсы
3. Найти рейс
4. Редактировать рейс
5. Удалить рейс
6. Агрегированные показатели
7. Сортировка рейсов
8. Выход
Введите выбор: █
```

Рисунок Б.1 – Главное меню приложения

Пункт 1-й:

```
gaukhar@MacBook-Air-Zhanet: ~/курсовая/котлин % cd "/Users/gaukhar/Downloads/курсовая" && kotlinc Mainn.kt -include-runtime -d Mainn.jar && java -jar Mainn.jar

== СИСТЕМА УЧЁТА АВТОБУСНЫХ РЕЙСОВ ==
Главное меню:
1. Ввод/вывод данных
2. Отобразить все рейсы
3. Найти рейс
4. Редактировать рейс
5. Удалить рейс
6. Агрегированные показатели
7. Сортировка рейсов
8. Выход
Введите выбор: 1

== Ввод / Вывод данных ==
1. Загрузить данные из CSV-файла
2. Сохранить данные в CSV-файл
3. Добавить новый рейс
Выберите действие: █
```

Рисунок Б.2 – 1-й пункт меню

```

gavrikna@nabbook: ~/Знаете курсовая Kotlin? & cd ~/Знаете/gra
&& kotlinc Mainn.kt -include-runtime -d Mainn.jar && java
1. Загрузить данные из CSV-файла
2. Сохранить данные в CSV-файл
3. Добавить новый рейс
Выберите действие: 3

=== Добавление нового рейса ===
Номер маршрута (например, 101): 333
Пункт отправления: Москва
Пункт назначения: Владимир
Время отправления (чч:мм): 08:30
Цена билета (руб.): 350
Общее количество мест: 50
Количество свободных мест: 30
Доступные статусы: Активен, Отменён, Задержан, Завершён
Статус рейса: Активен
Рейс успешно добавлен с ID = 1.

```

Рисунок Б.3 – Выбор 3 в первом пункте

```

=== Ввод / Вывод данных ===
1. Загрузить данные из CSV-файла
2. Сохранить данные в CSV-файл
3. Добавить новый рейс
Выберите действие: 2
Данные успешно сохранены в файл 'routes.csv'. Записей: 3.

=== СИСТЕМА УЧЁТА АВТОБУСНЫХ РЕЙСОВ ===
Главное меню:
1. Ввод/вывод данных
2. Отобразить все рейсы
3. Найти рейс
4. Редактировать рейс
5. Удалить рейс
6. Агрегированные показатели
7. Сортировка рейсов
8. Выход
Введите выбор: 1

=== Ввод / Вывод данных ===
1. Загрузить данные из CSV-файла
2. Сохранить данные в CSV-файл
3. Добавить новый рейс
Выберите действие: 1
Загружено 3 рейс(ов) из файла 'routes.csv'.

```

Рисунок Б.3 – Выбор 1 и 2 в первом пункте

2-й пункт меню:

```

===== СИСТЕМА УЧЁТА АВТОБУСНЫХ РЕЙСОВ =====
Главное меню:
1. Ввод/вывод данных
2. Отобразить все рейсы
3. Найти рейс
4. Редактировать рейс
5. Удалить рейс
6. Агрегированные показатели
7. Сортировка рейсов
8. Выход
Введите выбор: 2
===== Все рейсы =====

```

ID	Маршрут	Откуда	Куда	Время	Цена	Мест	Своб.	Статус
1	333	Москва	Владимир	08:30	350,00	50	30	Активен
2	202	Санкт-Петербург	Новгород	10:00	450,00	44	20	Активен
3	305	Казань	Нижний Новгород	14:15	280,00	55	55	Завершён

Рисунок Б.4 – 2-й пункт меню

3-й пункт меню:

```

===== СИСТЕМА УЧЁТА АВТОБУСНЫХ РЕЙСОВ =====
Главное меню:
1. Ввод/вывод данных
2. Отобразить все рейсы
3. Найти рейс
4. Редактировать рейс
5. Удалить рейс
6. Агрегированные показатели
7. Сортировка рейсов
8. Выход
Введите выбор: 3

===== Поиск рейсов =====
1. По пункту отправления
2. По пункту назначения
3. По номеру маршрута
4. По статусу
Выберите критерий: █

```

Рисунок Б.5 – 3-й пункт меню

```

===== Поиск рейсов =====
1. По пункту отправления
2. По пункту назначения
3. По номеру маршрута
4. По статусу
Выберите критерий: 1
Пункт отправления: Москва

```

ID	Маршрут	Откуда	Куда	Время	Цена	Мест	Своб.	Статус
1	333	Москва	Владимир	08:30	350,00	50	30	Активен

Рисунок Б.6 – Поиск по пункту отправления

=== Поиск рейсов ===
1. По пункту отправления
2. По пункту назначения
3. По номеру маршрута
4. По статусу
Выберите критерий: 2
Пункт назначения: Новгород

ID	Маршрут	Откуда	Куда	Время	Цена	Мест	Своб.	Статус
2	202	Санкт-Петербург	Новгород	10:00	450,00	44	20	Активен
3	305	Казань	Нижний Новгород	14:15	280,00	55	55	Завершён

Рисунок Б.7 – Поиск по пункту назначения

=== Поиск рейсов ===
1. По пункту отправления
2. По пункту назначения
3. По номеру маршрута
4. По статусу
Выберите критерий: 3
Номер маршрута: 305

ID	Маршрут	Откуда	Куда	Время	Цена	Мест	Своб.	Статус
3	305	Казань	Нижний Новгород	14:15	280,00	55	55	Завершён

Рисунок Б.8 – Поиск по номеру маршрута

=== Поиск рейсов ===
1. По пункту отправления
2. По пункту назначения
3. По номеру маршрута
4. По статусу
Выберите критерий: 4
Статусы: Активен, Отменён, Задержан, Завершён
Статус: Завершён

ID	Маршрут	Откуда	Куда	Время	Цена	Мест	Своб.	Статус
3	305	Казань	Нижний Новгород	14:15	280,00	55	55	Завершён

Рисунок Б.9 – Поиск по статусу

4-й пункт меню:

```
=== СИСТЕМА УЧЁТА АВТОБУСНЫХ РЕЙСОВ ===
Главное меню:
1. Ввод/вывод данных
2. Отобразить все рейсы
3. Найти рейс
4. Редактировать рейс
5. Удалить рейс
6. Агрегированные показатели
7. Сортировка рейсов
8. Выход
Введите выбор: 4

=== Редактирование рейса ===
Введите ID рейса: 1
Текущие данные:
-----
ID   | Маршрут | Откуда      | Куда      | Время | Цена   | Мест  | Своб. | Статус
-----
1    | 333     | Москва      | Владимир  | 08:30 | 350,00 | 50    | 30    | Активен
-----

Оставьте поле пустым для сохранения текущего значения.
Номер маршрута [333]: 104
Пункт отправления [Москва]:
Пункт назначения [Владимир]:
Время отправления [08:30]: 15:00
Цена билета [350.0]:
Общее количество мест [50]:
Свободных мест [30]: 25
Доступные статусы: Активен, Отменён, Задержан, Завершён
Статус [Активен]:
Рейс с ID=1 успешно обновлён.
```

Рисунок Б.10 – 4-й пункт меню

5-й пункт меню:

```
=== СИСТЕМА УЧЁТА АВТОБУСНЫХ РЕЙСОВ ===
Главное меню:
1. Ввод/вывод данных
2. Отобразить все рейсы
3. Найти рейс
4. Редактировать рейс
5. Удалить рейс
6. Агрегированные показатели
7. Сортировка рейсов
8. Выход
Введите выбор: 5

=== Удаление рейса ===
Введите ID рейса: 2
Рейс с ID=2 успешно удалён.
```

Рисунок Б.11 – 5-й пункт меню

```

===== СИСТЕМА УЧЁТА АВТОБУСНЫХ РЕЙСОВ =====
Главное меню:
1. Ввод/вывод данных
2. Отобразить все рейсы
3. Найти рейс
4. Редактировать рейс
5. Удалить рейс
6. Агрегированные показатели
7. Сортировка рейсов
8. Выход
Введите выбор: 2

===== Все рейсы =====

```

ID	Маршрут	Откуда	Куда	Время	Цена	Мест	Своб.	Статус
1	104	Москва	Владимир	15:00	350,00	50	25	Активен
3	305	Казань	Нижний Новгород	14:15	280,00	55	55	Завершён

Рисунок Б.12 – Отображение рейсов для проверки всех изменений

6-й пункт меню:

```

===== СИСТЕМА УЧЁТА АВТОБУСНЫХ РЕЙСОВ =====
Главное меню:
1. Ввод/вывод данных
2. Отобразить все рейсы
3. Найти рейс
4. Редактировать рейс
5. Удалить рейс
6. Агрегированные показатели
7. Сортировка рейсов
8. Выход
Введите выбор: 6

===== Агрегированные показатели =====
Всего рейсов: 2
Средняя цена билета: 315,00 руб.
Минимальная цена: 280,00 руб.
Максимальная цена: 350,00 руб.
Сумма всех цен: 630,00 руб.
Всего мест: 105
Свободных мест: 80

Количество рейсов по статусам:
Активен: 1
Завершён: 1

```

Рисунок Б.13 – 6-й пункт меню

7-й пункт меню:

Предварительно добавлены новые рейсы для того, чтобы показать сортировку рейсов

```

=== СИСТЕМА УЧЁТА АВТОБУСНЫХ РЕЙСОВ ===
Главное меню:
1. Ввод/вывод данных
2. Отобразить все рейсы
3. Найти рейс
4. Редактировать рейс
5. Удалить рейс
6. Агрегированные показатели
7. Сортировка рейсов
8. Выход
Введите выбор: 7

=== Сортировка рейсов ===
1. По ID
2. По номеру маршрута
3. По цене (возрастание)
4. По цене (убывание)
5. По количеству свободных мест
6. По статусу
Выберите поле: 3

```

ID	Маршрут	Откуда	Куда	Время	Цена	Мест	Своб.	Статус
4	333	Апрелевка	Москва	15:55	250,00	45	20	Активен
3	305	Казань	Нижний Новгород	14:15	280,00	55	55	Завершён
1	104	Москва	Владимир	15:00	350,00	50	25	Активен
6	555	Ногинск	Киров	23:05	985,00	45	30	Отменён
5	111	Тула	Калуга	00:30	1050,00	60	35	Задержан

Рисунок Б.14 – 7-й пункт меню и сортировка рейсов

8-й пункт меню:

```

=== СИСТЕМА УЧЁТА АВТОБУСНЫХ РЕЙСОВ ===
Главное меню:
1. Ввод/вывод данных
2. Отобразить все рейсы
3. Найти рейс
4. Редактировать рейс
5. Удалить рейс
6. Агрегированные показатели
7. Сортировка рейсов
8. Выход
Введите выбор: 8
Выход из программы...

```

Рисунок Б.15– Выход из программы

ПРИЛОЖЕНИЕ В

Тестовые сценарии

Сценарий 1. Загрузка данных из корректного файла при запуске приложения

Цель: проверить корректность загрузки данных из CSV-файла при запуске приложения.

Предусловие: файл routes.csv существует и содержит корректные сведения о рейсах.

Действия пользователя:

Запустить приложение.

Дождаться вывода меню.

Выбрать пункт «Отобразить все рейсы».

Ожидаемый результат: программа успешно загружает данные из файла и отображает список рейсов.

Сценарий 2. Загрузка данных при отсутствии файла при запуске приложения

Цель: проверить работу программы при отсутствии файла с данными.

Предусловие: файл routes.csv отсутствует в каталоге программы.

Действия пользователя:

Запустить программу.

Дождаться вывода меню.

Ожидаемый результат: программа не завершает работу аварийно, выводит меню и создаёт пустой список рейсов.

Сценарий 3. Просмотр всех рейсов

Цель: проверить корректность отображения списка рейсов.

Предусловие: в памяти программы содержатся записи о нескольких рейсах.

Действия пользователя:

Выбрать пункт меню «Отобразить все рейсы».

Ожидаемый результат: на экран выводится полный список рейсов с данными: ID, маршрут, откуда, куда, время, цена, места, статус.

Сценарий 4. Добавление нового рейса с корректными данными

Цель: проверить возможность добавления новой записи о рейсе.

Предусловие: программа запущена, коллекция доступна для изменения.

Действия пользователя:

Выбрать пункт «Ввод/вывод данных».

Выбрать «Добавить новый рейс».

Ввести номер маршрута — «101».

Ввести пункт отправления — «Москва».

Ввести пункт назначения — «Владимир».

Ввести время — «08:30».

Ввести цену — «350».

Ввести общее количество мест — «50».

Ввести свободных мест — «30».

Ввести статус — «Активен».

Ожидаемый результат: новый рейс успешно добавляется в список с автоматически присвоенным идентификатором.

Сценарий 5. Добавление нового рейса с некорректной ценой

Цель: проверить обработку ошибочного ввода числовых данных.

Предусловие: программа находится в режиме добавления рейса.

Действия пользователя:

Выбрать пункт «Ввод/вывод данных».

Выбрать «Добавить новый рейс».

Заполнить корректно первые поля.

В поле цены ввести отрицательное число или текст «abc».

Ожидаемый результат: программа выводит сообщение об ошибке, не завершает работу аварийно и возвращается в главное меню.

Сценарий 6. Добавление рейса с пустым полем

Цель: проверить обработку пустого обязательного поля.

Предусловие: программа находится в режиме добавления рейса.

Действия пользователя:

Выбрать пункт «Ввод/вывод данных».

Выбрать «Добавить новый рейс».

Оставить пустым поле пункта отправления (нажать Enter без ввода).

Ожидаемый результат: программа выводит сообщение об ошибке и завершает операцию добавления без создания записи.

Сценарий 7. Добавление рейса с недопустимым статусом

Цель: проверить валидацию поля статуса при добавлении.

Предусловие: программа находится в режиме добавления рейса.

Действия пользователя:

Выбрать пункт «Ввод/вывод данных».

Выбрать «Добавить новый рейс».

Заполнить все поля корректно.

В поле статуса ввести значение, отсутствующее в списке, например «Неизвестен».

Ожидаемый результат: программа выводит сообщение об ошибке, рейс не добавляется.

Сценарий 8. Редактирование данных по существующему идентификатору

Цель: проверить корректность изменения данных о рейсе.

Предусловие: в списке есть запись с указанным идентификатором.

Действия пользователя:

Выбрать пункт «Редактировать рейс».

Ввести существующий идентификатор рейса.

Изменить одно или несколько полей, например цену и статус.

Оставить остальные поля пустыми для сохранения.

Выбрать пункт «Отобразить все рейсы».

Ожидаемый результат: данные выбранного рейса успешно изменяются и отображаются в обновлённом виде.

Сценарий 9. Редактирование данных по несуществующему идентификатору

Цель: проверить поведение программы при вводе неверного идентификатора.

Предусловие: в списке нет рейса с указанным идентификатором.

Действия пользователя:

Выбрать пункт «Редактировать рейс».

Ввести идентификатор, отсутствующий в коллекции (например, 999).

Ожидаемый результат: программа выводит сообщение о том, что рейс с таким идентификатором не найден.

Сценарий 10. Удаление существующей записи

Цель: проверить корректность удаления рейса из коллекции.

Предусловие: в списке содержится запись о рейсе с указанным идентификатором.

Действия пользователя:

Выбрать пункт «Удалить рейс».

Ввести существующий идентификатор рейса.

Выбрать пункт «Отобразить все рейсы».

Ожидаемый результат: запись удаляется из коллекции и больше не отображается в общем списке.

Сценарий 11. Удаление записи по несуществующему идентификатору

Цель: проверить обработку удаления отсутствующей записи.

Предусловие: запись с введённым идентификатором отсутствует.

Действия пользователя:

Выбрать пункт «Удалить рейс».

Ввести несуществующий идентификатор.

Ожидаемый результат: программа выводит сообщение о том, что запись не найдена, и продолжает работу.

Сценарий 12. Поиск рейсов по существующему пункту отправления

Цель: проверить работу поиска по текстовому полю.

Предусловие: в списке есть рейсы с заданным пунктом отправления.

Действия пользователя:

Выбрать пункт «Найти рейс».

Выбрать критерий «По пункту отправления».

Ввести название города или его часть.

Ожидаемый результат: программа выводит все рейсы, соответствующие введённому запросу.

Сценарий 13. Поиск по отсутствующему значению

Цель: проверить поведение программы, если совпадения не найдены.

Предусловие: в списке отсутствуют рейсы с указанным пунктом отправления.

Действия пользователя:

Выбрать пункт «Найти рейс».

Выбрать критерий «По пункту отправления».

Ввести значение, отсутствующее в данных.

Ожидаемый результат: программа выводит сообщение о том, что рейсов с указанным значением не найдено.

Сценарий 14. Сортировка рейсов по цене

Цель: проверить корректность сортировки по числовому полю.

Предусловие: в списке есть несколько рейсов с разными ценами.

Действия пользователя:

Выбрать пункт «Сортировка рейсов».

Выбрать вариант «По цене (по возрастанию)».

Ожидаемый результат: записи выводятся в порядке возрастания цены билета.

Сценарий 15. Вычисление агрегированных показателей

Цель: проверить корректность вычисления статистики.

Предусловие: в программе загружено несколько записей рейсов.

Действия пользователя:

Выбрать пункт «Агрегированные показатели».

Ожидаемый результат: программа корректно вычисляет и отображает: общее количество рейсов, среднюю, минимальную и максимальную цену, сумму цен, общее и свободное количество мест, количество рейсов по статусам.

Сценарий 16. Сохранение данных в файл

Цель: проверить корректность сохранения данных.

Предусловие: в список были внесены изменения: добавление, редактирование или удаление рейсов.

Действия пользователя:

Выбрать пункт «Ввод/вывод данных».

Выбрать «Сохранить данные в CSV-файл».

Открыть файл routes.csv в текстовом редакторе.

Ожидаемый результат: файл routes.csv создан или перезаписан, содержит актуальные данные.

Сценарий 17. Ввод некорректного пункта меню

Цель: проверить устойчивость программы к ошибочному вводу.

Предусловие: программа находится в главном меню.

Действия пользователя:

Ввести букву, символ или номер, отсутствующий в списке пунктов меню (например, «abc» или «99»).

Ожидаемый результат: программа выводит сообщение о некорректном выборе и повторно предлагает ввести команду без завершения работы.